QUESTION 1

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("Dataset_Day8.csv")
# Columns where 0 is considered missing
missing_cols = ['Glucose', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction']

# Replace 0s with median (excluding zeros)
for col in missing_cols:
    median_val = df[df[col] != 0][col].median()
    df[col] = df[col].replace(0, median_val)
print(df[missing_cols].describe())
print("Missing data was found in four columns. These were replaced with median values, which is robust to outliers and maintains the central tendency.")
```

```
          Glucose  BloodPressure         BMI  DiabetesPedigreeFunction
count  768.000000     768.000000  768.000000                768.000000
mean   121.656250      72.386719   32.455208                  0.471876
std     30.438286      12.096642    6.875177                  0.331329
min     44.000000      24.000000   18.200000                  0.078000
25%     99.750000      64.000000   27.500000                  0.243750
50%    117.000000      72.000000   32.300000                  0.372500
75%    140.250000      80.000000   36.600000                  0.626250
max    199.000000     122.000000   67.100000                  2.420000
Missing data was found in four columns. These were replaced with median values, which is robust to outliers and maintains the central tendency.
```

QUESTION 2

```python
def remove_outliers_iqr(dataframe, columns):
    df_clean = dataframe.copy()
    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df_clean = df_clean[(df_clean[col] >= lower) & (df_clean[col] <= upper)]
    return df_clean

numeric_cols = df.columns.drop('Outcome')
df_clean = remove_outliers_iqr(df, numeric_cols)

# Show number of rows before and after outlier removal
print(f"Original dataset shape: {df.shape}")
print(f"Dataset shape after outlier removal: {df_clean.shape}")
print(f"Number of rows removed as outliers: {df.shape[0] - df_clean.shape[0]}")
print("Outliers can skew distance calculations in kNN. Removing them ensures better model stability and less variance in prediction.")
```

```
Original dataset shape: (768, 7)
Dataset shape after outlier removal: (698, 7)
Number of rows removed as outliers: 70
Outliers can skew distance calculations in kNN. Removing them ensures better model stability and less variance in prediction.
```

QUESTION 3

```
X = df_clean.drop('Outcome', axis=1)
y = df_clean['Outcome']

# Feature scaling is critical for distance-based algorithms
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42)

print("\n  Feature scaling applied using StandardScaler.")
print(f"Scaled feature matrix shape: {X_scaled.shape}")
print("\n Train-test split completed (70% train / 30% test):")
print(f"Training feature shape: {X_train.shape}")
print(f"Testing feature shape : {X_test.shape}")
print(f"Training labels shape : {y_train.shape}")
print(f"Testing labels shape  : {y_test.shape}")
print("Insight:\nStandardScaler ensures all features have mean=0 and std=1, which is essential for kNN's distance metric.")
print("The dataset is now split into 70% training and 30% testing, which balances training quantity and evaluation.")
```

```
Feature scaling applied using StandardScaler.
Scaled feature matrix shape: (698, 6)

 Train-test split completed (70% train / 30% test):
Training feature shape: (488, 6)
Testing feature shape : (210, 6)
Training labels shape : (488,)
Testing labels shape  : (210,)
Insight:
StandardScaler ensures all features have mean=0 and std=1, which is essential for kNN's distance metric.
The dataset is now split into 70% training and 30% testing, which balances training quantity and evaluation.
```

QUESTION 4a

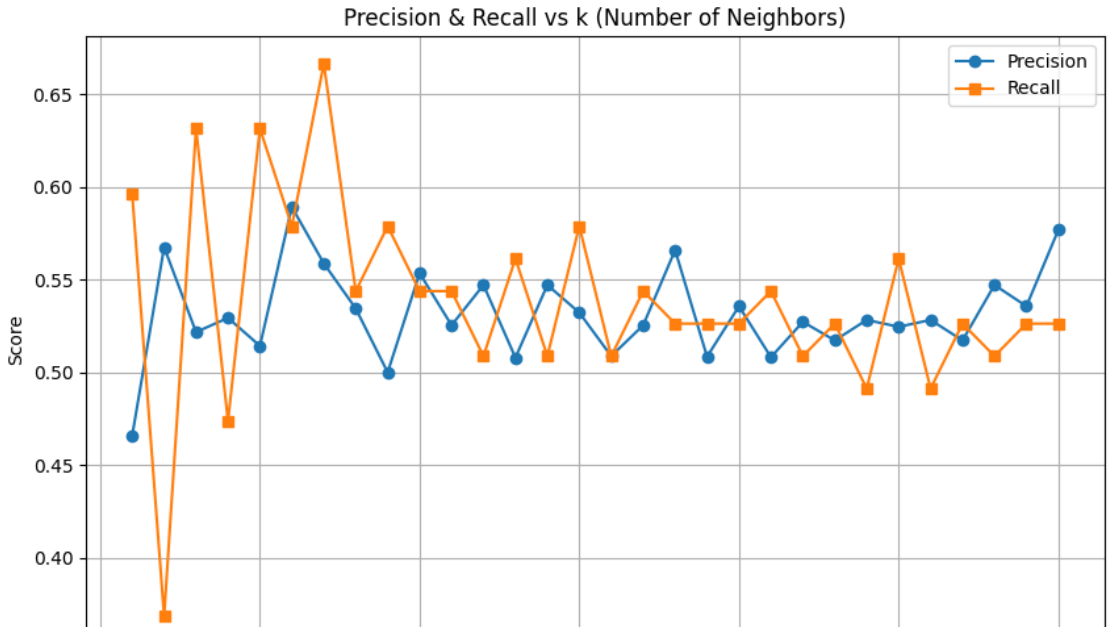
```python
X = df_clean.drop('Outcome', axis=1)
y = df_clean['Outcome']

# Feature scaling is critical for distance-based algorithms
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42)

print("Feature scaling applied using StandardScaler.")
print(f"Scaled features shape         : {X_scaled.shape}")
print("\n Data split into training and testing sets:")
print(f"Training features shape       : {X_train.shape}")
print(f"Testing features shape        : {X_test.shape}")
print(f"Training target labels shape  : {y_train.shape}")
print(f"Testing target labels shape   : {y_test.shape}")
print(f"Training set class balance    : {y_train.value_counts().to_dict()}")
print(f"Testing set class balance     : {y_test.value_counts().to_dict()}")
print("Scaling is essential for kNN to treat all features equally in distance computation.")
```

```
Feature scaling applied using StandardScaler.
Scaled features shape         : (698, 6)

 Data split into training and testing sets:

Training features shape       : (488, 6)

Testing features shape        : (210, 6)

Training target labels shape  : (488,)

Testing target labels shape   : (210,)

Training set class balance    : {0: 314, 1: 174}

Testing set class balance     : {0: 153, 1: 57}

Scaling is essential for kNN to treat all features equally in distance computation.
```

QUESTION 4b

```python
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
# Metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Default k=5 Model Performance:")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall   : {rec:.4f}")
print(f"F1 Score : {f1:.4f}")
print("These default metrics will act as a baseline. Further tuning can improve F1-score and precision depending on class imbalance.")

precisions = []
recalls = []
f1_scores = []
k_values = range(1, 31)

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)

    precisions.append(precision_score(y_test, y_pred))
    recalls.append(recall_score(y_test, y_pred))
    f1_scores.append(f1_score(y_test, y_pred))

# Plot
plt.figure(figsize=(10, 6))
plt.plot(k_values, precisions, label='Precision', marker='o')
plt.plot(k_values, recalls, label='Recall', marker='s')
plt.title('Precision & Recall vs k (Number of Neighbors)')
plt.xlabel('k')
plt.ylabel('Score')
plt.grid(True)
plt.legend()
plt.show()
```

```
Default k=5 Model Performance:
Accuracy : 0.7381
Precision: 0.5143
Recall   : 0.6316
F1 Score : 0.5669
These default metrics will act as a baseline. Further tuning can improve F1-score and precision depending on class imbalance.
```



QUESTION 4c

```python
for metric in metrics:
    for k in range(1, 21):
        knn = KNeighborsClassifier(n_neighbors=k, metric=metric)
        knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)
        f1 = f1_score(y_test, y_pred)

        if f1 > best_combo['f1']:
            best_combo = {'metric': metric, 'k': k, 'f1': f1}

print("\n Best kNN Configuration:")
print(f"Distance Metric: {best_combo['metric']}")
print(f"k: {best_combo['k']}")
print(f"F1 Score: {best_combo['f1']:.4f}")
print("Different metrics treat spatial relationships differently. Manhattan may outperform Euclidean in high-dimensional or skewed feature distributions." \
" Best F1-score indicates the most balanced model.")
```

```
 Best kNN Configuration:
Distance Metric: euclidean
k: 7
F1 Score: 0.6080
Different metrics treat spatial relationships differently. Manhattan may outperform Euclidean in high-dimensional or skewed feature distributions. Best F1-score indicates the most balanced model.
```